

Git-HOL 1

Git Configuration and Repository Setup

Aim

To configure Git, set up a local repository, integrate Notepad++ as the default editor, and perform basic Git operations.

Software Requirements

- Git Bash
- GitHub/GitLab Account
- Notepad++
- Windows Operating System

Procedure

1. Install Git Bash.
2. Configure the Git username and email.
3. Verify the Git configuration.
4. Integrate Notepad++ as the default Git editor.
5. Create a local repository named **GitDemo**.
6. Create a file named **welcome.txt**.
7. Add the file to the staging area.
8. Commit the file to the local repository.
9. Create a remote repository.
10. Pull from and push the local repository to the remote repository.

Git Commands Used

```
git config --global user.name "username"  
git config --global user.email "username@example.com"  
git init GitDemo  
git status  
git add welcome.txt  
git commit  
git pull origin master  
git push origin master
```

Sample Output

Initialized empty Git repository.

welcome.txt created successfully.

Changes committed successfully.

Working tree clean.

Changes pushed to remote repository.

Result

Git was successfully configured, a local repository was created, files were committed, and the repository was synchronized with the remote repository.

Git-HOL 2

Using .gitignore to Ignore Files and Folders

Aim

To ignore unwanted files and folders using the `.gitignore` file.

Software Requirements

- Git Bash
- Git Repository
- GitHub/GitLab

Procedure

1. Create a `.log` file.
2. Create a folder named `log`.
3. Create or edit the `.gitignore` file.
4. Add rules to ignore:
 - All `.log` files
 - The `log` directory
5. Save the `.gitignore` file.
6. Execute `git status`.

7. Verify that the ignored files are not displayed.

Example **.gitignore**

```
*.log  
log/
```

Git Commands Used

```
git status  
git add .gitignore  
git commit -m "Added gitignore"
```

Sample Output

On branch master

nothing to commit

working tree clean

Result

The **.gitignore** file was successfully configured to ignore unwanted log files and folders.

Git-HOL 3

Git Branching and Merging

Aim

To create a new branch, make changes, merge the branch into the master branch, and delete the branch.

Software Requirements

- Git Bash
- Git Repository
- GitHub/GitLab
- P4Merge Tool

Procedure

1. Create a new branch named **GitNewBranch**.
2. Switch to the new branch.
3. Create or modify files.
4. Commit the changes.
5. Check the repository status.
6. Switch back to the master branch.
7. Compare the branch with the master.
8. Merge the branch into the master.
9. View the commit history.
10. Delete the merged branch.

Git Commands Used

```
git branch GitNewBranch
git checkout GitNewBranch
git add .
git commit -m "Updated branch"
git checkout master
git merge GitNewBranch
git log --oneline --graph --decorate
git branch -d GitNewBranch
```

Sample Output

Branch 'GitNewBranch' created.

Switched to branch 'GitNewBranch'.

Merge completed successfully.

Deleted branch GitNewBranch.

Result

The branch was successfully created, merged with the master branch, and deleted after merging.

Git-HOL 4

Merge Conflict Resolution in Git

Aim

To resolve merge conflicts when multiple users modify the same file in different branches.

Software Requirements

- Git Bash
- Git Repository
- P4Merge Tool

Procedure

1. Verify that the master branch is clean.
2. Create a branch named **GitWork**.
3. Create a file named **hello.xml**.
4. Commit the changes.
5. Switch to the master branch.
6. Modify the same file with different content.
7. Commit the changes.
8. Compare the differences between branches.
9. Merge the branch into the master.
10. Resolve merge conflicts using the three-way merge tool.
11. Commit the resolved changes.
12. Update **.gitignore**.
13. Delete the merged branch.

Git Commands Used

```
git checkout -b GitWork
git add hello.xml
git commit -m "Added hello.xml"
git checkout master
git merge GitWork
git diff
git status
git commit
git branch -d GitWork
```

Sample Output

Auto-merging hello.xml

CONFLICT (content): Merge conflict in hello.xml

Conflict resolved successfully.

Working tree clean.

Result

The merge conflict was successfully resolved using Git's merge tools, and the updated changes were committed to the master branch.

Git-HOL 5

Cleaning Up and Pushing Changes to Remote Repository

Aim

To synchronize the local repository with the remote repository by pulling and pushing changes.

Software Requirements

- Git Bash
- GitHub/GitLab Repository

Procedure

1. Verify that the master branch is clean.
2. Display all available branches.
3. Pull the latest changes from the remote repository.
4. Push local commits to the remote repository.
5. Verify that the changes are reflected in the remote repository.

Git Commands Used

```
git status  
git branch  
git pull origin master  
git push origin master
```

Sample Output

Already up to date.

Enumerating objects...

Writing objects...

Everything up-to-date.

Changes successfully pushed to remote repository.

Result

The local repository was successfully synchronized with the remote repository by pulling the latest changes and pushing local commits.